

Attacking .NET deserialization - Alvaro Muñoz

Attacking .NET deserialization - Alvaro Muñoz - 1ns0mn1h4ck, YouTube.

- Published: 2018-04-28
- Original: <https://www.youtube.com/watch?v=eDfGpu3iE4Q>
- Preserved from: <https://www.youtube.com/watch?v=eDfGpu3iE4Q> (preserved-copy) on 2026-08-04
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source cited by ysonet, kept so the technique survives the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Attacking .NET deserialization - Alvaro Muñoz

- Video: <https://www.youtube.com/watch?v=eDfGpu3iE4Q>
- Channel: 1ns0mn1h4ck
- Published: 2018-04-28
- The media file itself is not archived; this is its metadata and transcript.

Description

Insomni'hack 2018 Title: Attacking .NET deserialization Speaker: Alvaro Muñoz

2016 was the year of Java deserialization apocalypse. Although Java Deserialization attacks were known for years, the publication of the Apache Commons Collection Remote Code Execution gadget (RCE from now on) finally brought this forgotten vulnerability to the spotlight and motivated the community to start finding and fixing these issues. .NET is next in line; formatters such as BinaryFormatter and NetDataContractSerializer are known to share similar mechanics which make them potentially vulnerable to similar RCE attacks. However, as we saw with Java before, the lack of RCE gadgets led some software vendors to not take this issue seriously. In this talk, we will analyze .NET serializers including third party JSON parsers for potential RCE vectors. We will provide real-world examples of vulnerable code and more importantly, we will review how these vulnerabilities were detected and fixed in each case.

Transcript

Retrieved with yt-dlp in a container, which is the only route YouTube still answers. Expect transcription errors in type and method names.

[Music] okay so yeah my name is Abram North I'm also known as pond tester in Twitter and github and social networks I work as a principal security researcher with 45 basically in the static code analyzer team I don't consider myself a speaker but I happen to present at different conference conferences that I like black hat and DEF CON and so on and I have a special interest in remote code execution vulnerabilities through our web in web applications so I basically successfully exploited products and vendors like Microsoft what I call workday a Salesforce and so on actually the Microsoft one one of the Microsoft ones I'm going to present today so apart from that I have some experience around serialization back in 2012 I was looking at different REST API implementation libraries like Restylane and Andres DC and I was able to find some remote code execution vulnerabilities by abuse in Access dream and XML decoder and then in 2013 I started looking at Java binary visualization right at that time and to my knowledge this was the first public remote restitution gadget available for Java digitization but unfortunately no one was paying attention to javadi's realization at those days so it was not until 2015 when Gabriel Lawrence presented the gadget in Apache Commons collection that people started looking at Java visualization vulnerabilities I started fixing them and finding them and so on so that's why I'm saying unfortunately because we could have saved ourselves a couple of years by starting looking at that in 2012 at least so at that time when they released discard yet I started looking in judicialization again I was able to find multiple remote code execution cages was able to bypass the standard mitigation technique which is called as look-ahead object visualization and I was able to find the cut yet in the Java Runtime which means that you don't require a third-party libraries to be present in the classpath in order to exploit these vulnerabilities so then I moved to J so this visualization which I presented last year at black hat and DEF CON but most of these is is Java right so at some point I changed my interest and I started looking at dotnet and this is what this talk is about um we're going to see a brief introduction into serialization formats and then we will see what are the most popular and used dotnet serialize are both by Microsoft and third-party libraries like JSON dotnet if we have time we will see some some demos including this Microsoft promote Constitution in nature and then we will move to see how we can detect find these vulnerabilities and then fix them so serialization 101 so I don't know why but in English the Americans I guess and British people use pickle for serialization to me pickle is just this green vegetable but it's actually very famous like the Python libraries called pickle this talk up by crisp Rahab was called Marceline pickles but well to me it make a great excuse to use a Rick and Morty drawing see me in my slides and well this is Rick obviously and if what you have an instance of Rick that is living in your VM and at some point you want to put this this object into a different representation so you can store it in the file system for example or maybe put it into a database or send it across the network or whatever so you put it through the magic portal and then you get pickle Rick right and this is the sterilization format of Rick at a later stage you want you may want to recover this instance of Rick in your VM so you basically reconstruct the object based on the

information on the serialized format so the the thing that is key for serialization attacks here is that we are not sending just the pickle but we are sending also this pickle Rick right and this is called the type discriminator the type discriminator is nothing else that the class name or the type name that you are sending along with their data and so if you are familiar with Java serialization for examples you are probably familiar with well this is the binary format and this is basically the ASCII representation of the class name and basically the server is going to use these class names to actually instantiate and reconstruct the objects at the server side so that's Java serialization in dotnet for example in JSON dotnet you have something similar but this is JSON format right so you have type attribute here that contains something that looks like toad net assembly dotnet type name so if the attackers are able to change those type discriminators then they will be able to force an instantiation of an arbitrary class in during the initialization process and that will basically allow the attacker to instantiate a multi instant instance instead of the require so there is nothing really dangerous on getting the server to instantiate or call the default constructor for a different or arbitrary type because the four constructors are just parameter lists and they don't do anything paths normally but normally in order to fully reconstruct the object there are some callbacks and some methods that may get invoke right so for example if you are familiar with Java serialization you have the read object and read resolve callbacks that are invoked in order for I mean this allow the developers to fully reconstruct the object so maybe you need to initiate initialize something or register the object in a jndi context or I don't know notify someone about the initialization or something like that so in dotnet we also have this kind of callbacks we have the serialization constructor we have some serialization callbacks like on on serialization for example and we also have some annotations that we can use to let the framework know which method should be invoke within the serialization process apart from the binary format normally in JSON and XML and the setters are also invoked right so basically we invoke the default constructor we reconstruct the object and then we invoke the setters in order to populate the values to the different properties of the of the object so with that we fully reconstruct the object but there is some code that is executed so gadgets gadgets are basically different types that are sitting in the classpath of the application that is this aerial icing you can provide a type name but you cannot provide a type byte code so these types are special because if they are provided with some malicious values they can lead to arbitrary code execution or to something bad like removing files or doing something bad so this is a simple example that is a gadget a setter gadget so it starts with I'm poking some setters it's in dotnet in the class system windows data object data provider and I like to use these for explaining what our gadget is because it's very simple right so you have three different setters that when you invoke them with attacker control data they will basically initiate a chain of calls that will end up invoking the invoke method on instance write this one down here so basically there we are we are using reflection to invoke an arbitrary method on an arbitrary object instance passing some arbitrary method parameters and those arbitrary values all come from the different setters right like the value that you pass to the setter so this is an example of a payload for JSON Dannette basically what we are saying here is ok we are going to deserialize an object data provider so the server is going to instantiate the object that are provided by calling the default constructor and then it's

going to invoke the setters for the object instance the method parameters and the method name and because we saw that these setters will eventually call their reflection code to invoke the method then we will be able to get a rhetorical execution if we can provide this piece of JSON to a vulnerable JSON the net endpoint so with that I think that's enough about gadgets I wrote this application called why so silly adore net there is a Java version that will basically generate payloads for you the Java one is just for binary format sorry for them Java binary format this one supports different federalizes so you have to pass the format there that you want your data to be your payload to be serialized so for example you have to specify what cut you do do you want to use what format a do you want to use and what command do you want to invoke okay so with that let's move to the dotnet CLI service part basically this is nothing new back in the days when I was playing with the rest endpoints in 2012 at Jane's four so from the Google project zero was presenting at blackhat that some malicious code could be executed as the result of dotnet visualization so at that time he found that binary formatter and net data contract serializer both could lead to some dangerous execution but at that point there was no remote code execution gadgets right so it happened like in Java the Java visualization vector it was a very well known vector since at least 2007 or something like that but until 2015 when the Apache Commons collection gardius remote code execution gadget was published no one was actually paying attention and actually started fixing things so same thing here it was known that by 94 matter was dangerous but it was not until last year when a colleague of mine Alex Miller Alex Mills and James Forester and myself published three different gadgets for remote execution when people started looking at fixing the dead civilization so we extended the list by James 4 so by looking at other side Eliezer's so James already said that binary format that is realized is vulnerable in the default configuration but binary format there is also used by all therefore matters right so for example the binary message for matter is used by the Microsoft message queue the equivalent to activemq for example in Java so if you are capable of sending arbitrary message to a message system and they will be using binary message format that is as vulnerable as the binary formatter one also object the state formatter that is the one that is used to sterilize the viewstate data in the in asp.net it's using an object state for matter so because of that we will be able to get arbitrary code execution in view estates if we know how to modify the view estate without breaking the signature that we will see that later so James already told no told us about that data contracts analyzer but also shop for Mattel using web services fast Jason and sweet Jason third-party libraries they are also vulnerable in the default configuration so this is what I think is the most popular and use vulnerable pattern in dotnet that is basically take some objects and then sterilize them using Bey binary formatter and then basics before encoding that I'm putting that into a cookie right that means that when the cookie get backs to the server the application is going to the satellites that data and there is no H Mac here there is no nothing that prevents the attacker to from modifying that and injecting the payload so I first find this vulnerability in a site or a company called app harbor that is basically like a Roku for donate applications and if you browse the application you will find that at some point we're assigning some cookie with some values that it starts with a a a a D which is the magic number for dotnet serialization which made them basically vulnerable so we reported the issue they fix it and then we found that

they even have a blog post explaining how to do that in your own applications like you can use the cookie temp data provider with binary format there and then make your application vulnerable in that way so now this blog post is deleted but I found that this same advice was everywhere is like really and tens of blog post is in India trapping different lists and also it was used by Microsoft in an old version of the asp.net MVC framework so at some point in 2013 probably after the James foresaw talk they remove this cookie temp data provided from the asp.net MVC framework but I think that the damage was already done many frameworks and many developers were already using this pattern in their applications as we will see so simple example well this is a vulnerability that I found in Microsoft Asia Network basically you can expose your intranet right this box here is your intranet and then expose your web applications to the remote workers right here by using what is known as the application proxy service and that means that you will install a connector in your network and then your employees will basically access a URL that is provided by Microsoft and then they will be redirected to the Active Directory login that will basically set your credentials in your own Active Directory and if everything is correct you will be given a token that you will provide to the application proxy server and then you will finally authenticate to the web application right so I'm pretty sure that you're familiar with this login page so basically I don't know if you can read anything from them so I can I can't stop it apparently okay well you can't read anything but this is the URL provided by Microsoft you can change it and put your any domain you want and then I do access your remote application you are redirected to Microsoft login right you don't actually have to login into Microsoft and this is a pre authentication remote code execution vulnerability what I'm doing here is basically I'm just deleting the cookies for this page before sending the request to burp so I'm going to do that right now and basically what I will do is send this request to the repeater and the response is basically the redirection to the Microsoft login page and then a cookie there Adam I don't think that you can read it but it's basically starting with this magic number again this aad so I basically just well if we take this value and we take it to the decoder to see what is inside of this basic string or blob we will see that there are some type discriminators and type names so this is basically using dotnet serialization specifically it's using binary formatter so at that time I was I haven't written the why so she got net so I used my own gut yet generator at this point I'm basically what I'm going to execute is the nslookup to my own DNS server and basically I'm going to accelerate the name of the of the computer name the computer name variable to verify and to prove Microsoft that I was executed in that in there servers are not in the Internet servers so that takes a while because the VM was very slow that day if I try that locally I will get a connection from from my own servers what you can't read anything but there is a moon of a that is my own computer name and now if I copy paste the payload that is generated and that you can generate now with white Sociedad net if you copied out to the request and send it to the application proxy remember we are not even authenticating so this is pre authentication remote code execution in AC or network we just copy that we change to our DNS server and as soon as we send a request we will see that it's been executed in the Microsoft servers so we get like four different resolutions right there and well you cannot read it but this is not moon Ave that is my computer name this is Microsoft come and those are Microsoft APM IPS so with that I will if you send a request

again you get the connection from different servers which means that these application proxies are working like in a load balancer way which means that if you compromise the application proxies you can now get access to all the Internet's that are being proxied by the server so with that those those were the four matters that were vulnerable in the default configuration right like binary format you should never use that with untrusted data there are a different set of four matters that are vulnerable if you configure them in a special way or if you let the users provide not just the data to be this realized by but something else that is called the specter type so for example if you use data contract serializer or XML serialize this are sterilizers that are okay to handle and untrusted input and trusted data but you should never use or never let the users to control the expected type so that's exactly what DotNetNuke that is a content management system or actually the most popular content management system in in.net was actually doing so basically again I don't know if you can read anything but there is a cookie that contains this XML item that comes from an unsigned cookie that the attacker can control and then this XML we get the type name extracted from the XML and then we initialize the XML serialization a m-- so now the attacker can control the expected type and by expected type I mean that for example they are going they're only going to work with that expected type right so for example if you are sending or expecting a shopping cart you cannot send an object data provider like the cat yet that I saw you beforehand you can only send and deserialize a shopping cart and then at a different point they they sterilized the rest of the cookie and then this CMS was been able to remote code execution I don't have time to show you the video but is in the DEF CON talk if you want to check it and then there is a different set of visualizers that are vulnerable depending on the configuration of the VC réaliser so for example JavaScript serialize er that is the native solution for JSON serialization in dotnet the one that is used for example by asp.net you should never use that with her type resolver if you initialize this JavaScript sterilizer with a type resolver that basically means that it is going to include the type discriminators the type information with the json data and then it's going to process that and that is going to allow the attackers to control which types are going to be reconstructed and then that will lead to an arbitrary code execution so that's JavaScript steriliser json dotnet is the most popular json library for.net has the same problem in the default configuration is secure but if the Velo pairs enable this type name handling setting and they put anything older than than the known value so for example type name handling objects or out or arrays that means that the type information will be included in the json data and that will allow the attackers to actually get arbitrary code execution so that's actually what a framework that is called breeze was actually doing so breeze is a framework for entity management database entity management so that what they do is basically the expose your database entities through a REST API right right so for example they expose a rest endpoint to add a new record or to delete a record to modify a record and so on and they do that automatically so they have this demo application for developers to learn about the framework and well this is a very simple application that allows you to handle on manage a different car models so for example you can change the name of a model and submit it and it will be persisted in your database if we send that to a through burp we can actually change for example they in the name of the car model and if you can change that and send it to repeater you will see or I don't

know if you will see but there is an empty dictionary in here that is called say option and an empty object or an empty dictionary because this is using JSON dotnet with the type name handling what I'm doing here is basically using the object that I provided gadget that I showed you at the beginning of the talk remember the one that was using the three setters to call any method and I was basically using that to invoke the process dot start and invoke the calculator so you copy/paste these directly into your request and now if we go back to the server now we can check that there is no calculator running as soon as we send a request it will be dis a realized the object that I provided will be instance here that the setters will be invoked and the calculator will be executed so that was about a brief introduction to some of the NC realizers now how can we find them right how can we find wearable endpoints so one simple way is by inspecting the traffic as I said this is the magic number for binary formatter and this is probably the most used one if you search in Solem for it you will find like five or six hundred instances is still alive then what I wrote a very simple plugin for purp that I released today is very simple it doesn't I doesn't mean to be very accurate so it has some false positives because this magic number can appear in images for example or it may report a view state that contains serialize data but then is assign it so you cannot change that without a server realizing but it detects different by a magic numbers not just a binary for matter 1 and different encodings not just by 64 so it's kind of useful to detect probabilities like the azure one or any some others you can also use an active approach like black box approach sending a payload that if gets the serialize will make the server to sleep for some seconds or to initiate a DNS resolution that you can detect in your server or you can use is like instrument your application instrument your VM and see if some untrusted data reaches some of the this visualization endpoints if you have access to the source code like I did for example for DNA or for breeze frameworks you can actually run a static analysis and look for untrusted data flowing into one of these things like for example single data flows are very simple to identify for binary for matter or for example for XML serialization twill required one flowing into the data to be the serialize and another one flowing into the expected type so again it's simple most of the static analysis tools should support that and now at the third part of the talk is how to fix vulnerable endpoints or how to verify if the vendors are actually applying good fixes so the first way is to stop using civilization I mean this seems like rather stupid but I've found many cases some of them I cannot disclose yet but that they are using civilization to cellulite strings right one case that I can talk about because it's already fixed is in the Nancy framework which is a web application framework and instead what they have these instances are cookie for CSR protection but instead of putting the token directly into the cookie value they were sterilizing the string into an object and then basically for encoding that data into the cookie which may basically make them vulnerable to remote code execution instead of CSR so I don't know also if you don't really need type discriminators don't enable them right so that was the case for example for the breeze framework that I showed you once that I reported the issue they it took them like 12 hours to fix the issue because they realized that they were not actually using the type information right when they wrote the framework for the first time the clients were meant to be done F doesn't have clients so they were exchanging some types and some dotnet civilization but at some point they added a JavaScript client and because of that they stopped using the type

information internet because it was useless but no one disabled that and then they leave their different work vulnerable so this is the way of protecting Jason dotnet for example like JSON dotnet sorry like a bridge framework did basically just explicitly called time name handling and then past and non-value to make sure that the data is not the type information is not included in the data so the first approach was to stop using it the second approach is to verify that no one is tampering with your cookies for example so if you do that always you use H Mac never use something like md5 or sha-1 with the secret and the data because that's gonna be able to house extension attacks and this is the way that for example app Harbor or AC or fix the issue right they just included a signature an h-back so they were able to verify that the client was not actually verified changing or tampering with the data so that should be the case like for example if you cannot stop using this allocation and you have to put something into the client-side because maybe I don't know for performance reasons or whatever you have to sign it and make sure that no one is touching that as I said at the beginning and that's what Microsoft did with the asp.net implementation of the cookie temp data provider in the first version of the framework it was vulnerable but then they fix it using the data protection API and then they also fix the asp.net view state that we will see later so it's very simple I just use the data protector API protect the bytes and that will basically sign and encrypt the data using the web.config encryption on validation keys so view state is an interesting case at the beginning like many years ago they were not signing or encrypting the view state so the view state is basically a representation of the state of a given page so instead of keeping that server-side they send it to you and then when you request or send a and their request back to the server they take that data and reconstruct the previous state of the page so as I said they are using object state for matter with which is vulnerable to remote code execution attacks so if you allow the client to change the view state that will lead to remote code execution so at some point they included the option to enable an H mark and encryption to the view state because well at that point there were at that point in time there were no remotes and gadgets but if you could change the viewer state you could change the application logic and lead to different bugs so in 4.5.2 they even ignored the enable view estate max so even if you put that into false the asp.net is going to ignore that and it's always going to encrypt and validate and include the H Mac for the view state they even back ported that to 1.1 so if you are not ignoring the patches you should be protected so when I was writing this slide I was curious about how many pad set servers were in the net in the network and basically I checked with southern and out of the 10,000 results that they gave me 200 were born apples so this made like a big interesting I don't know probably Bitcoin miners botnet anyway so at some point they improve the cryptographic posture of the viewstate right because if you are able to leak the crypto at the encryption and validation keys you can actually use that to change the view state and then it will lead to remote code execution so Microsoft decided to add some additional protection by using a key derivation function and something that they call the purpose and the purpose is nothing else that some strings right so I don't know if you can read it but there is a primary purpose and some specific purposes so when you visit a URL and they generate four different strings and these four strings will be used to actually derive a new set of keys so instead of using the encryption and validation keys that you have in your web config file they will derive

some keys that are unique for the specific URL that you are visiting so for example if you visit the account register URL the first purpose is always going to be the same is just describing the purpose of the encryption that is going to be client state meaning that is going to be used for the view state and that's not going to change but now the second string is something that is called a slash account right this seems like rather predictable given the URL that you are visiting the third one is account underscore register underscore.aspx which again is rather predictable and also easy to guess and the fourth one is this abuse state user key which contains something that looks like an arbitrary unique token like seven seven something but now if you go and check the cookies for this page that you are visiting you will see the anti cross-site request forgery token it's actually using the same value as the viewstate so now you have all the different pieces that you need to derive the new keys so and get arbitrary code execution still you need to be able to leak the encryption and validation key this is the key point here so the whole idea of these slides is to remark that you need to be very careful with not leaking the keys how can you lick the keys well if you have our neck sexy moon ability for example or maybe if you haven't patched these very old CVS like 2010 2015 that allowed users to actually lick the web.config file from Troy's hand I found that apparently I haven't seen that myself but apparently there are some error pages that can leak the encryption and validation keys and last but not least in github you will find hundreds of keys so if you are lucky enough that someone is deploying the application with the very same keys then you may be lucky and get arbitrary code execution so the final recommendation here is to use encryption to you to get a is to encrypt your web config file at least some sections of the web config file so even if you get like an X X availability in your application the attackers won't be able to get the encryption invalidation key and be able to actually escalate from xxe to remote code execution so we talked about the stop using it signing it and then binding it is the third approach so this is a very dangerous approach but is well it should work but you have to be very careful at implementing it basically the idea is to constrain which types are going to be allowed you did indeed during the serialization right so you can basically implement a Silurian binder implement the bind to type method and then you are provided the assembly type the assembly name and the typename so you can check them right before this type get instantiated and then decide if you want to continue with the visualization or just discard it so if you go and use this way I don't know if you can read that in red but never use black lists they are very easy to bypass I will show you now the bypass code yet and never use a very broad white list approach I will show you an example now so this is an example of a good security allowed list serialization binder which basically is implementing the serialization binder interface and basically is checking for a specific strict list of allowed classes and if you have the classes that are allowed are not found it's throwing an exception right you should never return null because if you're returning null then it will default to the default binder and then it will allow the exploit to go on so once you have your civilization binder you have to assign it to the binder property and then DC allies and trusted data that's the other way of using a serialization binder so as I said never use a broad whitelist approach this is actually save the code from a Microsoft very popular Microsoft product I'm in the process of this closing this vulnerability so I cannot tell the name of the product but the code looks very similar to this right and at

some point they are checking if the type starts with Microsoft dot put the name of a very popular Microsoft Word here and if that's the case then they will allow you to continue with the serialization because they think or they trust that they don't have remote code execution gadgets in their name in their namespace in in their own classes and maybe that's true and they don't have this client this kind of cut yet in their namespace but what is probably sure is that they contain bypass gadgets so what is our bypass cut yet basically it's a type that initiates and nested this realization from additional ization constructor right so you are protecting the outer visualization and maybe you are binding that and checking that these malicious claw are not used are not digitalized but then you are allowing the visualization of this class that it's going to initiate an inner or nested isseul ization which basically won't be using any binders so it will allow the visualization of untrusted malicious remote code execution gadgets so again in your DC allocation binders never use or if you are checking for the protections maybe doing some bug hunting if you see that they are using is a signal for from or if you see for example that they are returning new that means that this binder can be bypassed also if they are using reflection in their binder it's very easy to get at the OS and us the application so that was the third approach binding it the fourth approach is replacing it right so maybe you don't really need to send all these type information along with the data so maybe if the client and the server can agree that they are going to exchange I don't know I use her instance maybe just send in the name the address the last name and so on it's enough and you don't have to send that they are using combed acme dot user or any any type name so that's the approach that Google protocol buffers use that's a good approach if you want to DC lies and trusted data last but not least JSON and XML are also secure but as long as you don't include type information like we saw for example in JSON Dannette so that's all I have for you so if you have any questions any question someone thank you for talk um I was wondering did you try to report all those machine keys because you can do quite a few things with those machine keys from github or did you find any issues in modern or in popular software so I haven't reported any of them there are literally hundreds of them in github and I guess that for example if these are open source projects or open source frameworks that the people that are going to use those frameworks are going to change those those keys otherwise if you know that a specific framework is using those keys and you can look for it websites using those frameworks and try to exploit that but yeah I don't know we try we probably for the most popular one we should try to find if they are using those hard-coded keys and report that that actually makes a very good CTF task right so being able to lick the keys then use the key derivation purposes and all the stuff so maybe for the next year okay so no more questions so okay thank you for being there and thank you for your talk thank you [Applause]